Experiment No. 9

Title: Development of a Personalized AI Assistant using an Open-Source Large Language Model, Retrieval-Augmented Generation (RAG), and Fine-Tuning

Aim: To develop a personalized AI assistant using an open-source foundational language model, integrate a Retrieval-Augmented Generation (RAG) system to answer questions from user-specific documents, and investigate the effect of parameter-efficient fine-tuning using LoRA or QLoRA on the model's response quality.

Objectives:

1. To understand the working principle of Large Language Models (LLMs).
2. To develop an AI assistant using an open-source pretrained language model.
3. To implement a RAG pipeline for retrieving information from user-specific documents.
4. To understand document chunking, embeddings, and vector databases.
5. To explore parameter-efficient fine-tuning using LoRA or QLoRA.
6. To compare the responses of the base model, RAG-enabled model, and fine-tuned model.
7.

Pre-Requisites:

1. Basics of Python programming.
2. Basic understanding of Generative AI and foundation models.
3. Familiarity with Google Colab or Jupyter Notebook.
4. Basic knowledge of audio formats and text prompts.

Theory:

A Large Language Model (LLM) is a deep learning model trained on large amounts of text to understand and generate human-like language. Open-source LLMs can be downloaded and used for tasks such as question answering, summarization, and conversational assistance.

A general-purpose LLM may not know the contents of a user's private documents. A personalized AI assistant can address this limitation by combining an LLM with Retrieval-Augmented Generation and, optionally, fine-tuning.

Key concepts:

- Large Language Model (LLM): Generates responses based on the input prompt and learned language patterns.
- Retrieval-Augmented Generation (RAG): Retrieves relevant information from an external knowledge source and provides it to the LLM as context.

- Document Chunking: Divides large documents into smaller sections for efficient retrieval.
- Embeddings: Represents text as numerical vectors that capture semantic meaning.
- Vector Database: Stores embeddings and supports similarity-based retrieval.
- Fine-Tuning: Further trains a pretrained model on selected examples to adapt its behavior.
- LoRA/QLoRA: Parameter-efficient fine-tuning methods that reduce the number of model parameters that need to be updated or the memory required for training.

In a RAG system, documents are divided into chunks and converted into embeddings. When a user asks a question, the system retrieves relevant chunks and supplies them to the LLM to generate a grounded response.

LoRA or QLoRA can be used to adapt a model using a suitable training dataset. RAG and fine-tuning serve different purposes: RAG supplies relevant external information, while fine-tuning adapts model behavior or task performance.

Conclusion:

The experiment demonstrates the development of a personalized AI assistant using an open-source LLM. Students learn how RAG enables document-based question answering and how LoRA or QLoRA can adapt a pretrained model. The experiment also highlights the differences between retrieval-based knowledge access and model fine-tuning.